In [3]:
```
list1=[20,22,24]
list2=['Ramesh','Suresh','Vignesh']

# Ramesh age is 20
# Suresh age is 22
# Vignesh age is 24

for i in range(len(list2)):
    print(f"{list2[i]} age is {list1[i]}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

zip

- zip()
- iterate through zip
- how many file we zipped = that many values

In [9]:
```
for i,j in zip(list1,list2):
    print(f"{j} age is {i}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

In [5]:
```
reversed('python')
```

Out[5]:
```
<reversed at 0x2233ebd1c60>
```

In [ ]:
```
list1=[20,22,24]
list2=['Ramesh','Suresh','Vignesh']

# attachement ===== dictionary
```

In [ ]:
```
'',[],{}
```

In [ ]:
```
{key:value}

key-value pair
```

In [12]:
```
dict1={'Ramesh':20,
    'Suresh':22,
    "Vignesh":24}
```

In [13]:
```
dict1
```

Out[13]:
```
{'Ramesh': 20, 'Suresh': 22, 'Vignesh': 24}
```

In [14]:
```
type(dict1)
```

Out[14]:
```
dict
```

In [ ]:
```
int

str

float

bool

complex

list

dict
```

In [ ]:
```
dict1={'Ramesh':20,
       'Suresh':22,
       "Vignesh":24}

# key:   different data types
# value: different data types
```

In [16]:
```
dict2={'Fruite':"Apple",
       'Suresh':22,
        24:"Vignesh",
        30.5:"age",
        25:25}
dict2
```

Out[16]:
```
{'Fruite': 'Apple', 'Suresh': 22, 24: 'Vignesh', 30.5: 'age', 25: 25}
```

In [18]:
```
dict3={"Fruites":["Apple","Banana"]}
dict3
```

Out[18]:
```
{'Fruites': ['Apple', 'Banana']}
```

In [19]:
```
# key as list of values
dict4={["Apple","Banana"]:"Fruites"}
dict4
```

```
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[19], line 1
----> 1 dict4={["Apple","Banana"]:"Fruites"}
      2 dict4

TypeError: unhashable type: 'list'
```

In [20]:
```
# Key as tuple
dict5={('a','b','c'):26}
dict5
```

Out[20]:
```
{('a', 'b', 'c'): 26}
```

In [21]:
```
# key as dictionary
dict6={{"apple":25}:'Buy'}
dict6
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last
t)
Cell In[21], line 2
      1 # key=dictionary
----> 2 dict6={{"apple":25}:'Buy'}
      3 dict6

TypeError: unhashable type: 'dict'
```

In [22]:
```
dict7={True:"true"}
dict7
```

Out[22]:
```
{True: 'true'}
```

In [23]:
```
dict8={10+20j:"true"}
dict8
```

Out[23]:
```
{(10+20j): 'true'}
```

In [ ]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}   # works

dict2={'Fruite':"Apple",'Suresh':22,24:"Vignesh",30.5:"age",25:25}  # Works

dict3={"Fruites":["Apple","Banana"]}   # Works

dict4={["Apple","Banana"]:"Fruites"}  # Fail

dict5={('a','b','c'):26}              # Works

dict6={{"apple":25}:'Buy'}            # Fail

dict7={True:"true"}                  # Works

dict8={10+20j:"true"}                # Works
```

In [24]:
```
dict9={"Buy":{"apple":25}}
dict9
```

Out[24]:
```
{'Buy': {'apple': 25}}
```

In [25]:
```
dict10={"Ramesh":20,"Ramesh":20}
dict10

# Duplicates are not allowed
```

Out[25]:
```
{'Ramesh': 20}
```

In [26]:
```
# Keep key names as same , give the different values
dict11={"Ramesh":20,"Ramesh":22}
dict11

# Latest value updated
```

Out[26]:
```
{'Ramesh': 22}
```

In [27]:
```
dict12={20:"Ramesh",22:"Ramesh"}
dict12
```

Out[27]:
```
{20: 'Ramesh', 22: 'Ramesh'}
```

In [ ]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}   # works

dict2={'Fruite':"Apple",'Suresh':22,24:"Vignesh",30.5:"age",25:25}  # Works

dict3={"Fruites":["Apple","Banana"]}   # Works

dict4={["Apple","Banana"]:"Fruites"}  # Fail

dict5={('a','b','c'):26}              # Works

dict6={{"apple":25}:'Buy'}            # Fail

dict7={True:"true"}                  # Works

dict8={10+20j:"true"}                # Works

dict9={"Buy":{"apple":25}}           # Works

dict10={"Ramesh":20,"Ramesh":20}    # Duplicates are not allowed

dict11={"Ramesh":20,"Ramesh":22}    #  latest value will take

dict12={20:"Ramesh",22:"Ramesh"}    # Keys are different so two values
```

In [28]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
```

In [ ]:
```
- type   dict

- len     3

- max     wrt only key value

- min     wrt only key value

- sum     ---we can do sum if key as numbers------------

- sorted   ascii

- reversed  keys
```

In [29]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
type(dict1)
```

Out[29]:
```
dict
```

In [30]:
```
len(dict1)
```

Out[30]:
```
3
```

In [36]:
```
max(dict1),min(dict1)
```

Out[36]:
```
('Vignesh', 'Ramesh')
```

In [35]:
```
ord('R'),ord("S"),ord("V")
```

Out[35]:
```
(82, 83, 86)
```

In [37]:
```
sum(dict1)
```

```
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[37], line 1
----> 1 sum(dict1)

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [38]:
```
dict2={200:20,300:22,400:24}
sum(dict2)
```

Out[38]:
```
900
```

In [39]:
```
sorted(dict1)
# List of keys in sort order
```

Out[39]:
```
['Ramesh', 'Suresh', 'Vignesh']
```

In [41]:
```
sorted({'z':26,"a":1,"M":13})

# A=65
# a=97
```

Out[41]:
```
['M', 'a', 'z']
```

In [45]:
```
for i in reversed(dict1):
    print(i)
```

```
Vignesh
Suresh
Ramesh
```

In [ ]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
type(dict1)                              # dict

len(dict1)                               # 3

max(dict1),min(dict1)                    # Vignesh, Ramesh     ASCII

sum(dict1)                               # Sum will not applicable if key as s

dict2={200:20,300:22,400:24}
sum(dict2)                               # Sum will works if key as numbers

sorted(dict1)                            # Sorted based on ASCII

sorted({'z':26,"a":1,"M":13})

for i in reversed(dict1):                # Reverse
    print(i)
```

in

In [50]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
'Ramesh' in dict1
'Suresh' in dict1
'Vignesh' in dict1

for i in dict1:
    print(i)

# if you iterate dictionary using for loop
# the output is key only
```

```
Ramesh
Suresh
Vignesh
```

index

In [53]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1['Ramesh'],dict1['Suresh']

# in dictionary keys and values are together
# if you want to retrive the values
# you can do by using keys only
```

Out[53]: (20, 22)

In [ ]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1['Ramesh']  # 20
dict1['Suresh']  # 22
dict1['Vignesh'] # 24

print(dict1[i])
```

In [63]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
for i in dict1:
    print(i,dict1[i])
```

```
Ramesh 20
Suresh 22
Vignesh 24
```

Concatenation

In [56]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict2={"Saresh":26}
dict1+dict2

# dictionary concatenation is not possible
```

```
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[56], line 3
      1 dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
      2 dict2={"Saresh":26}
----> 3 dict1+dict2

TypeError: unsupported operand type(s) for +: 'dict' and 'dict'
```

In [60]:
```
'a'+'b',[1]+[2],(1)+(2)

{}+{}
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last
t)
Cell In[60], line 3
      1 'a'+'b',[1]+[2],(1)+(2)
----> 3 {}+{}

TypeError: unsupported operand type(s) for +: 'dict' and 'dict'
```

In [67]:
```
dict1={'Fruites':['Apple','Banana','Cherry']}
# Retrive the cherry

# by using key, i will get values
# in side value , cherry is there
# that value is a list
# index operations start
dict1['Fruites'][2]
```

Out[67]: 'Cherry'

In [72]:
```
dict1={'Fruites':
       ['Apple','Banana',{'rate':25}]
      }
# retrive the 25
dict1['Fruites'][2]['rate']
```

Out[72]: 25

In [73]:
```
dict1={{{{{"Salary":100000}}}}}
dict1
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last
t)
Cell In[73], line 1
----> 1 dict1={{{{{"Salary":100000}}}}}
      2 dict1

TypeError: unhashable type: 'dict'
```

In [81]:
```
dict1={'salary':{'1lakh':{'do':[{'ds':'4months'},
                                'de',
                                'mle']
                          }
                }
      }
dict1['salary']['1lakh']['do'][0]['ds']
```

Out[81]: '4months'

methods

In [82]:
```
dir({})
```

Out[82]:
```
['__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__ior__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__ne__',
 '__new__',
 '__or__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__ror__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'clear',
 'copy',
 'fromkeys',
 'get',
 'items',
 'keys',
 'pop',
 'popitem',
 'setdefault',
 'update',
 'values']
```

In [ ]:
```
'clear',
'copy',
'fromkeys',
'get',
'items',
'keys',
'pop',
'popitem',
'setdefault',
'update',
'values'
```

keys-values-items

In [87]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
keys=dict1.keys()
keys
```

Out[87]:
```
dict_keys(['Ramesh', 'Suresh', 'Vignesh'])
```

In [85]:
```
type(keys)
```

Out[85]:
```
dict_keys
```

In [89]:
```
# the type is dict_keys
# it is look like list but not list
# index operations are not works
# convert into list
list(keys)
```

Out[89]:
```
'Ramesh'
```

In [ ]:
```
# practice dictionary
# keys
# values
# items
# copy
# clear
# strings assignment
```

In [ ]:
```
how to make dictionary
what are the data types for keys and values
type len max min sum
in
index will not work
to retrive value we need to use keys
keys  values  items
```

In [4]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1.keys()
# data type is  dict keys
# it look like list , but is not a list
# list methods are not applicable
# we are converting into list data type
list(dict1.keys())
```

Out[4]:
```
['Ramesh', 'Suresh', 'Vignesh']
```

In [6]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
list(dict1.values())
```

Out[6]:
```
[20, 22, 24]
```

In [7]:
```
# Ramesh age is 20
# Suresh age is 22
# Vignesh age is 24

# Method-1: assume we have two lists
names=['Ramesh', 'Suresh', 'Vignesh']
age=[20, 22, 24]
for i,j in zip(names,age):
    print(f"{i} age is {j}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

In [10]:
```
# Method-2: assume we have a dictionary
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
for i in dict1:
    print(f"{i} age is {dict1[i]}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

In [13]:
```
# Method-3 :  keys and values in two dictionary
# take a dictionary
# extract two lists keys and values
# perporm the operation

dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
names=list(dict1.keys())
age=list(dict1.values())
for i,j in zip(names,age):
    print(f"{i} age is {j}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

In [ ]:
```
################## M-1 ####################
names=['Ramesh', 'Suresh', 'Vignesh']
age=[20, 22, 24]
for i,j in zip(names,age):
    print(f"{i} age is {j}")

################ M-2 ####################
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
for i in dict1:
    print(f"{i} age is {dict1[i]}")

################ M-3 ##########################
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
names=list(dict1.keys())
age=list(dict1.values())
for i,j in zip(names,age):
    print(f"{i} age is {j}")
```

In [14]:
```
# by using items method
# Ramesh age is 20
# Suresh age is 22
# Vignesh age is 24

dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1.items()
```

Out[14]:
```
dict_items([('Ramesh', 20), ('Suresh', 22), ('Vignesh', 24)])
```

In [24]:
```
for i,j in dict1.items():
    print(f"{i} age is {j}")
```

```
Ramesh age is 20
Suresh age is 22
Vignesh age is 24
```

In [30]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
list(dict1.items())
```

Out[30]:
```
[('Ramesh', 20), ('Suresh', 22), ('Vignesh', 24)]
```

In [22]:
```
a,b='Ramesh',20
print(a)
print(b)
```

```
Ramesh
20
```

In [29]:
```
f_m1='sanjay','sanjay father'
f_m1
```

Out[29]:
```
('sanjay', 'sanjay father')
```

In [17]:
```
num=1,2,3,4,5
num
```

Out[17]:
```
(1, 2, 3, 4, 5)
```

In [19]:
```
def add(n1,n2):
    return(n1,n2,n1+n2)

n1,n2,summ=add(50,60) # 50,60,110
out  # (50,60,110)
```

Out[19]:
```
(50, 60, 110)
```

In [38]:
```
# WAP ask the user provide a 5 random numbers between 10 to 50
# find it is an even number or odd number
# output={'even':[10,20,30],'odd':[25,35]}

# step-1: create an empty dictionary
# step-2: create two empty lists even_list odd_list
# step-3: for loop 5 times
# step-4:    get the random number
# step-5:    if <condition>:
# step-6:        append into list
# step-7:    else:
# step-8:       append into odd list
# step-9: create a dictinary

import random
dict1={}
even_list,odd_list=[],[]
for i in range(5):
    num=random.randint(10,50)
    if num%2==0:
        even_list.append(num)
    else:
        odd_list.append(num)

dict1['even']=even_list
dict1['odd']=odd_list
dict1
```

Out[38]:
```
{'even': [16, 36], 'odd': [35, 11, 27]}
```

In [55]:
```
string1='can can you canner can as you can a can type of canner'
# how many times can repaeted
# output:dict1={'can':5,'you':2 ,'canner':2.................}

# step-1: create an empty dic
# step-2: split the string ==== > list of items
# step-3: iterate over the list === > each element will come
# step-4: count method
# step-5: create a dictionary
str1="can can you canner as you can a can type of cancer"
d={}
str1_list=str1.split(" ")
for i in str1_list:
    count = str1_list.count(i)
    d[i]=count

dict(sorted(d.items()))

# sorted is a key word
```

Out[55]:
```
{'a': 1,
 'as': 1,
 'can': 4,
 'cancer': 1,
 'canner': 1,
 'of': 1,
 'type': 1,
 'you': 2}
```

In [56]:
```
str1="can can you canner as you can a can type of cancer"
d={}
str1_list=str1.split(" ")
for i in sorted(str1_list):
    count = str1_list.count(i)
    d[i]=count

d
```

Out[56]:
```
{'a': 1,
 'as': 1,
 'can': 4,
 'cancer': 1,
 'canner': 1,
 'of': 1,
 'type': 1,
 'you': 2}
```

In [48]:
```
str1="can can you canner as you can a can type of cancer"
list1=str1.split()
list1.count('can'),str1.count('can')
```

Out[48]:
```
(4, 6)
```

In [42]:
```
string1="can can you canner can as you can a can type of canner"
wordlist=string1.split()
dict1={}
for i in range(len(wordlist)):
    dict1[wordlist[i]]= string1.count(wordlist[i])
print(dict1)
```

```
{'can': 7, 'you': 2, 'canner': 2, 'as': 1, 'a': 9, 'type': 1, 'of': 1}
```

In [43]:
```
dict1={}
word_list=string1.split()
for word in word_list:
    count=word_list.count(word)
    dict1[word]=count
print(dict1)
```

```
{'can': 5, 'you': 2, 'canner': 2, 'as': 1, 'a': 1, 'type': 1, 'of': 1}
```

In [44]:
```
string1 = "can can you canner can as you can a can type of canner"
words = string1.split()
word_counts = {}
for word in words:

    if word in word_counts:
        word_counts[word] += 1
    else:
        word_counts[word] = 1

print("Word Counts:")
print(word_counts)
```

```
Word Counts:
{'can': 5, 'you': 2, 'canner': 2, 'as': 1, 'a': 1, 'type': 1, 'of': 1}
```

In [33]:
```
dict1={}
dict1['Ramesh']=20
dict1['Suresh']=22
dict1['Ramesh']=30
dict1
```

Out[33]:
```
{'Ramesh': 30, 'Suresh': 22}
```

In [ ]:
```
dict2={"Ramesh":20,'Suresh':22}
```

In [64]:
```
list1=['hyd','Mumbai','Chennai','Bengaluru']
out=[]
dict1={}
for i in list1:
    if len(i)>3:
        out.append(i)
dict1['city']=out
dict1['city']
dict1.values()

dict1={}
dict1['city']=[i for i in list1 if len(i)>3]
dict1
```

Out[64]:
```
{'city': ['Mumbai', 'Chennai', 'Bengaluru']}
```

pop-popitem-del

In [69]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
# remove specified key and return the corresponding value.
dict1.pop("Vignesh")
# Removing the Vignesh and 24
```

Out[69]:
```
24
```

In [70]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1.pop()
```

```
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call las
t)
Cell In[70], line 2
      1 dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
----> 2 dict1.pop()

TypeError: pop expected at least 1 argument, got 0
```

In [72]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
# remove specified key and return the corresponding value.
dict1.popitem()

#Remove and return a (key, value) pair as a 2-tuple.  (a,b)

#Pairs are returned in LIFO (last-in, first-out) order.
dict1
```

Out[72]:
```
{'Ramesh': 20, 'Suresh': 22}
```

In [75]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
del dict1
dict1  # available or not you already deleted
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call las
t)
Cell In[75], line 3
      1 dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
      2 del dict1
----> 3 dict1

NameError: name 'dict1' is not defined
```

In [76]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
del dict1['Ramesh']  # Pop
dict1
```

Out[76]:
```
{'Suresh': 22, 'Vignesh': 24}
```

- pop requires key as argument, it will return the value and remove the key value
- pop item does not require any argument, by default it will remove the last key:value
- del is a keyword you can delete entire dictionary or you can delete specific key:value

Set as default

In [84]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
#Insert key with a value of default if key is not in the dictionary
#Return the value for key if key is in the dictionary, else default.
dict1.setdefault("Bunny")
# Im inserting a key: Bunny ,
# we are not providing any value by default: None

#-----------------------------------------------
dict1.setdefault("Chinni",25)
dict1['Rani']=35
#-----------------------------------------------

# Ramesh is already exised , the value will not change
dict1.setdefault('Ramesh',50)  # not change
dict1['Ramesh']=50             # will change to 20 to 50

# ----------- Bunny: None    None into 30--------------
dict1.setdefault('Bunny',50)   # will not work
dict1
```

Out[84]:
```
{'Ramesh': 50,
 'Suresh': 22,
 'Vignesh': 24,
 'Bunny': None,
 'Chinni': 25,
 'Rani': 35}
```

In [85]:
```
dir({})
```

Out[85]:
```
['__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__ior__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__ne__',
 '__new__',
 '__or__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__ror__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'clear',
 'copy',
 'fromkeys',
 'get',
 'items',
 'keys',
 'pop',
 'popitem',
 'setdefault',
 'update',
 'values']
```

In [ ]:
```
- clear  copy

- keys  values  items

- pop  popitem del

- setdefault
```

get

In [1]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
# get ==== retrive some thing
# key:value
# if i want to retrive , i can retrive only value
# that by using key
dict1['Ramesh']
```

Out[1]: 20

In [4]:
```
dict1.get('Ramesh',50)
```

Out[4]: 20

In [5]:
```
dict1.get('Rameshh')
```

In [ ]:
```
dict1.popitem()
```

update

In [8]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
d2={'Chinni':25}
dict1.update(d2)
dict1

#ValueError: dictionary update sequence element #0 has length 6; 2 is requi
```

Out[8]: {'Ramesh': 20, 'Suresh': 22, 'Vignesh': 24, 'Chinni': 25}

In [11]:
```


# list extend is same as update in dictionary
l1=[1,2,3]
l2=['A','B','C']
l1.extend(l2)  # [1,2,3,'A','B','C']
l1
```

Out[11]: [1, 2, 3, 'A', 'B', 'C']

In [15]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict2={"Bunny":30}
dict2.update(dict1)
dict2
```

Out[15]:
```
{'Bunny': 30, 'Ramesh': 20, 'Suresh': 22, 'Vignesh': 24}
```

In [ ]:
```
# what is mean by iterable:
any thing you can print using for loop
```

In [18]:
```
for i in dict1:
    print(i)
```

```
Ramesh
Suresh
Vignesh
```

In [19]:
```
dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
dict1.update([1,2,3])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 2
      1 dict1={'Ramesh':20,'Suresh':22,"Vignesh":24}
----> 2 dict1.update([1,2,3])

TypeError: cannot convert dictionary update sequence element #0 to a sequence
```

In [22]:
```
l1=['A','B']
l1.extend([1,2,3])
l1
```

Out[22]:
```
['A', 'B', 1, 2, 3]
```

In [23]:
```
l1=['A','B']
l1.extend('python')
l1
```

Out[23]:
```
['A', 'B', 'p', 'y', 't', 'h', 'o', 'n']
```

In [24]:
```
dir({})
```

Out[24]:
```
['__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__ior__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__ne__',
 '__new__',
 '__or__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__ror__',
 '__setattr__',
 '__setitem__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'clear',
 'copy',
 'fromkeys',
 'get',
 'items',
 'keys',
 'pop',
 'popitem',
 'setdefault',
 'update',
 'values']
```

In [ ]:
```
# fromkeys: home work
```

In [ ]:
```
# strings
# list
# dictionary ====== interview questions
```

In [ ]:
```
find the prime number 1 to 100 store in a list
list vs tuple

140Qns book===== 140 qns
```

In [ ]:
```
class note books
read that text book
assignment qns
140qns : interview
```

In [ ]:
```
tuple write up

- how to read the tuple     # comments

- difference between tuple and list

- a=1,2,3    a=(1,2,3)

- type  max  min  sum sorted reversed

- in

- index  mutable vs immutable

- compare this with list

- for loop  in and index

- tuple methods
```

In [25]:
```
a=(1,2,3)
type(a)
```

Out[25]:
```
tuple
```

In [26]:
```
dir(a)
```

Out[26]:
```
['__add__',
 '__class__',
 '__class_getitem__',
 '__contains__',
 '__delattr__',
 '__dir__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getnewargs__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__init__',
 '__init_subclass__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__rmul__',
 '__setattr__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'count',
 'index']
```

In [ ]: